

ATML - Assignment 0

Task-1

1. The ResNet-152 baseline

— take a pretrained Resnet-152, adopt its final classifier to a smaller dataset such as CIFAR-10,

- Freeze the backbone
- train only the new classification head
- record performance

Resnet-152 → roughly 152 layers
↓
Residual Network

$$y = H(x) + x$$

↳ basically there are 2 ways to learn this

1. We put input x into the model and get $H(x)$ and add that to input to get y

$$2. y - x = H(x)$$

↓
if we subtract ^{input} output from output, we know what model should give us, ^{what we get} $H(x)$

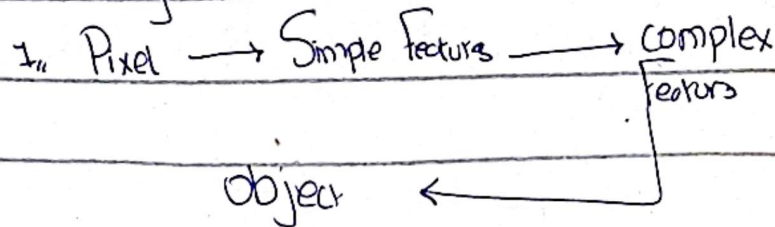
↓
Usually we backpropagate

CIFAR-10 $\rightarrow$ a very small 32×32 pixel color image

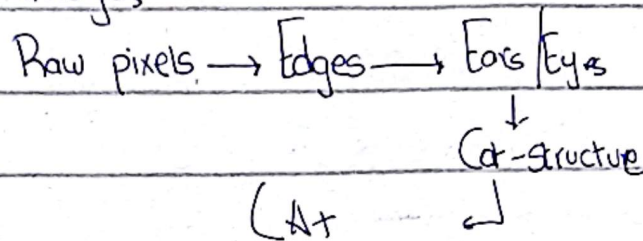
$\hookrightarrow 32 \times 32 \times 3$
width height colors - RGB

CNN - Convolutional Neural Networks

A CNN basically learns



eg. A cat image,



Q. Why convolutional NN?

$\rightarrow$ in normal NN.

Suppose CIFAR-10 image, $32 \times 32 \times 3$

$\rightarrow 3072$ pixel

Flatten it becomes $[pixel_1, pixel_2, \dots]$

in on image, location matters
also

$\hookrightarrow$ Convolution $\rightarrow C \rightarrow CNN$

$\hookrightarrow$ use small matrices called

$\hookrightarrow$ filter

$\hookrightarrow$ kernel

→ Now, this Filter is moved around, how?

Imagine, we have a image ~~and~~ of 64×64 and a kernel 3×3 . it starts from top of an image, takes dot product to get a value. After that, it moves a spot right and do the same and does it till it reaches bottom of the image. The scores are then called a Feature Map

↓

To get accurate Features, it predicts, calculates loss and backpropagate to correct the kernel

→ Many CNN layers stacked, helps ~~with~~ becomes overpowered

→ After Feature maps are made, then they are passed through activation function ReLU,

↓

0 → -ve

make non-linearity

↳ can help bending, gating, decision boundaries

→ Then, they are passed through 2x2 max pooling.

↓
extracting a part of Feature, and then take its strongest Feature

↳ 64 Feature maps → just smaller

↓
Then, another convolution layer, till we get high-level Feature maps

↓
Convert Feature maps into Feature vector of $[1, x]$

Suppose I have,

$[4, 4, 512]$ → For each in 512, take average to

make a vector of $[1, 512]$

↓
Classifier → take the highest score and choose it as to classify

Task 2

Revision

VIT

Vision Transformer

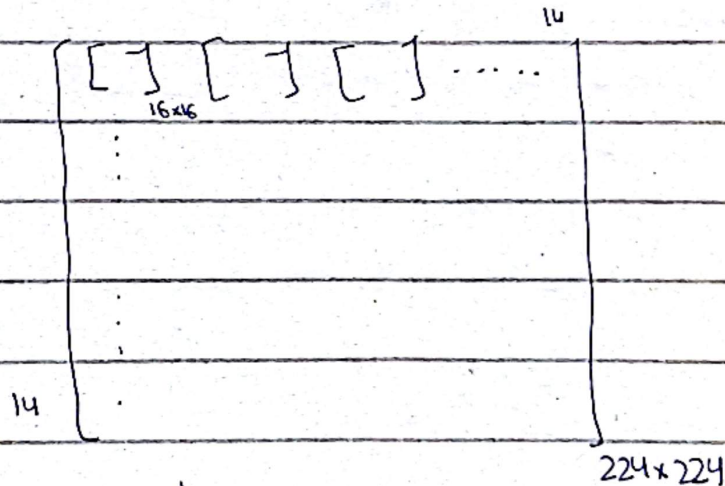
↓

→ chop image into patches, like tokens, and then communicate them using tokens → self attention

↳ Suppose image is 224×224

↳ VIT uses 16×16 patches

↓
14 patches



↓

↳ so there are a total of 196 patches

↓

Also, a CLS token

$[CLS], P_1, \dots, P_{196}$

to

Embedding $\rightarrow$ Patch

$\downarrow$
Flatten $\rightarrow$ Linear Projection $\rightarrow$ Embedding

Positional embedding $\rightarrow$ position of patch

VIT input $\rightarrow$ Patch embedding + Positional embedding

$\Rightarrow$ CLS token:

used For Classification

$\downarrow$
basically after passing through transformer, whatever the value is in CLS is used to classify that image

$\Rightarrow$ Self-attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

$\Rightarrow$ Each head is passed through multi-head and they grab different things

basically what I remember from

and in task 2.2.

we have an image ~~20~~ and its 197 patches

Those patches are converted into transformer tokens

So, each ~~token~~ picture has 197 tokens.

All 197 tokens are passed through each attention

head. Inside a head, each token calculates its

attention towards all 197 tokens, giving a

197×197 matrix.

↳ After ~~averaging~~ all attention heads,

we get $[1, 197, 197]$.

Then, we take the CLS row, and ~~the~~ remove

the CLS token, remaining its score towards the

196 image patches

CLIP

Contrastive Language-Imaging Pretraining

2 separate encoders

text image



turns everything into vector

Cat image → vector 1

Cat text → vector 2

} must be similar

Encoder:
takes input
and convert it into
a
smaller representation
that the
model can work
with

→ The individual numbers usually don't have an interpretation

→ Instead, the position of other vectors compared to the
vectors contains semantic information

↓
Representative Space / Embedding Space

→ Cosine Similarity =

$$\cos(x, y) = \frac{x^T y}{\|x\| \|y\|}$$

$\|x\| \|y\|$

1

0

-1

↓
Same direction

↓
unrelated

↓
Opposite
direction

InfoNCE

Normally, CIFAR-10 images $\rightarrow$ Train $\rightarrow$ Classifier $\downarrow$ Predict

Zero-Shot Classifier $\Rightarrow$ Takes label $\downarrow$ Encode it $\downarrow$ image comes $\rightarrow$ check its cosine similarity $\downarrow$ argmax $\rightarrow$

$\rightarrow$ Contrastive learning:

Not only it attracts the correct answer, it should repel all the incorrect options too

$\hookrightarrow$ it $\rightarrow$ image embedding

$\rightarrow$ L2 normalization:

$$\hat{x} = \frac{x}{\|x\|}$$

$\downarrow$
after normalization $\|x\| \rightarrow 1$

$\downarrow$
So it remains in a unit sphere

and

$$\cos(x, y) = x^T \cdot y$$

~~Modality~~

Modality gap :

modality $\rightarrow$ a type of data
 $\downarrow$
text

images

$\rightarrow$ no matter, how much similar ~~are~~ text and image is,
globally the 2 modalities may occupy somewhat
different regions.

$\Rightarrow$ Clip-embedding is like $(50, 512)$
 $\downarrow$
we can't draw them

so

T-SNE $\Rightarrow (50, 512) \rightarrow (50, 2)$

$\downarrow$
thus plotting them.

= t-SNE takes high-dimensional representations
and create lower-dimensional ~~structure~~
visualization that tries to preserve useful
neighbor-hood structure.

→ Orthogonal Procrustes:

$$\min_R \|XR - Y\|_F$$

→ basically, suppose image embedding and text embeddings are of some structure but at different angles

→ Procrustes:

rotate 1 embedding, so it lines with other

⇒ * orthogonal transformation preserve geometry

$\|XR - Y\|_F \rightarrow$ how different are these matrix overall

$$\|A\|_F \rightarrow \sqrt{a^2 + b^2 + c^2 + d^2}$$

$$A \rightarrow \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

⇓

Rotate X using R , compare it again Y , and calculate the total mismatch

⇓

We want that R that makes mismatch as small as possible

SVD $\rightarrow$ Single Value decomposition

$$A = U \Sigma V^T$$

$\downarrow$

way of decomposing matrix into components representing useful geometric transformation.

→ VAE on MNIST

↓
Auto encoder

↓
encoder = compresses image

↓
decoder = decompresses image

Compressed image / representation $\Rightarrow$ Latent representation
latent vector

→ Latent Space:

$d_z = 2$

↓
meaning every image is compressed and
represented by 2 numbers

↓
plotting them gives latent space

VAE $\rightarrow$ creates a smooth, organized latent space

→ A normal encoder might say:

$$x \rightarrow z \rightarrow [1.2, -0.1]$$

→ A VAE says,

don't give me an exact location

Give me a probability distribution describing
where the image should exist

$$q(z|x) = \mathcal{N}(\mu, \sigma^2)$$

Suppose hand written 1 has,

latent representation $\rightarrow (2, 3)$

$$\downarrow$$
$$\mu = (2, 3)$$

$\sigma \rightarrow \text{Var}$

$\downarrow$
how spread possible
locations are

$\therefore \text{VAE} \rightarrow \text{probabilistic}$

$$\rightarrow \log \sigma^2$$

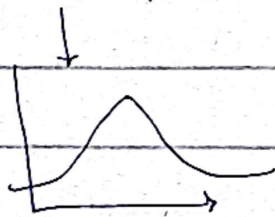
$\rightarrow$ basically neural network output any number
from $-\infty$ to ∞

$\downarrow$
but σ^2 must be positive

so,

$$\sigma^2 = \exp\left(\frac{1}{2} \log \sigma^2\right)$$

$$\rightarrow Z \sim N(\mu, \sigma^2)$$



$$\Rightarrow \star Z \sim N(\mu, \sigma^2)$$

$\downarrow$
Creates problem for backpropagation

$$E \sim N(0, 1)$$

$\Downarrow$

$$Z = \mu + \sigma \cdot E$$

Compress image

Encoder

μ

log var

σ

Random E

$$Z = \mu + \sigma E$$

Decoder

decompresses & reconstruct that image

→ Pixel values (x_i) → $0 < x_i < 1$
↓ ↓
might be might be
black white

⇒ model each pixel using bernoulli output something
or use final sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

↓
[0, 1]

⇒ VAE loss:

VAE → constructs image
calculate loss

For MNIST → dataset,

↳ if reconstructions look shit, loss ↑

$$q_{\theta}(z|x) = N(\mu, \sigma^2)$$



we want this close to $N(0,1)$

→ KL Divergence:

$$D_{KL} = \frac{1}{2} \sum_j \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right)$$

$$\mathcal{L}_{ELBO} = \mathbb{E}_{q_{\phi}(z|x)} \left[\log p_{\theta}(x|z) \right] - D_{KL}(q_{\phi}(z|x) \| p(z))$$

$$\mathcal{L}_{ELBO} = \mathbb{E}_{q_{\phi}(z|x)} \left[\log p_{\theta}(x|z) \right] - D_{KL}(q_{\phi}(z|x) \| p(z))$$

↓
Reconstruction loss

↓
KL loss